

Tabular Classification

LIN TONG, G2608825L

IN6227-Assignment-1

Variant-2

INTRODUCTION

This report presents an end-to-end tabular classification workflow applied to the dataset (31,112 rows, 15 features). The target variable is label with two classes: no and yes. The training distribution is no=23,645 and yes=7,464, giving a 3.168:1 imbalance. The data contains 50 missing feature values and three missing target labels; rows with missing labels were removed before training. There are seven numeric and eight categorical predictors, with no duplicate rows. The objective is to build a leakage-safe, reproducible pipeline and compare transparent linear and nonlinear classifiers under the same evaluation protocol.

METHODS OR PROCEDURES

All learned preprocessing is embedded inside an sklearn Pipeline and ColumnTransformer, so imputation, scaling, and encoding are refit within each training fold. Numeric columns use median imputation followed by standard scaling; categorical columns use most-frequent imputation followed by one-hot encoding with handle_unknown="ignore". All 15 original features are retained. No feature is selected using the test set, and no additional feature engineering is imposed because the data already includes the derived composite_rank field. Four classifiers are trained: DummyClassifier with a stratified strategy as the no-information baseline, LogisticRegression for an interpretable linear benchmark, RandomForest for nonlinear interactions, and GradientBoosting as a strong tabular baseline. Stratified five-fold cross-validation with shuffle=True and random_state=42 preserves the class ratio. Mean CV F1 is the primary selection metric; ROC-AUC and PR-AUC provide additional ranking evidence. The selected model is retrained on all training rows and evaluated once on the held-out test set. Balanced class weights are used for LogisticRegression and RandomForest because the minority class is about one quarter of the data. The model settings were kept deliberately modest: LogisticRegression used max_iter=1000 with class_weight=balanced; RandomForest used 200 trees with class_weight=balanced; GradientBoosting used 100 estimators; and DummyClassifier used the stratified strategy. I did not tune on the test set. The only stopping condition for the iterative logistic model was

convergence before max_iter; the tree ensembles used their fixed estimator counts. This makes the comparison reproducible, although a future version could perform a small grid search inside the training folds.

RESULTS

Test-set performance for the selected RandomForest is accuracy=0.8272, precision=0.6177, recall=0.7156, F1=0.6631, ROC-AUC=0.8887, and PR-AUC=0.7010. The CV comparison is: DummyClassifier accuracy=0.635 and F1=0.235; LogisticRegression accuracy=0.790 and F1=0.660; RandomForest accuracy=0.827 and F1=0.664; GradientBoosting accuracy=0.839 and F1=0.626. The test confusion matrix contains 8,762 true negatives, 1,403 false positives, 901 false negatives, and 2,267 true positives. The figure shows that the model identifies most minority-class cases while retaining a moderate false-positive count.

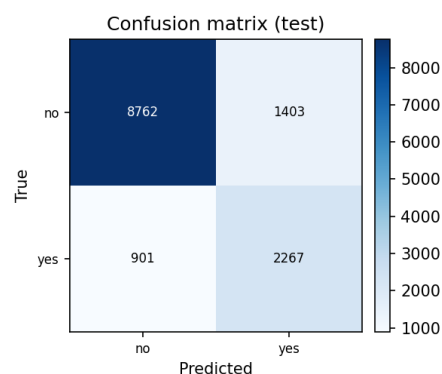


Figure 1. Test-set confusion matrix.

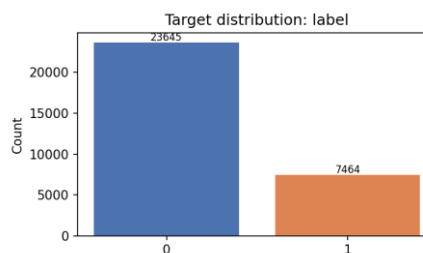


Figure 2. Target class distribution.

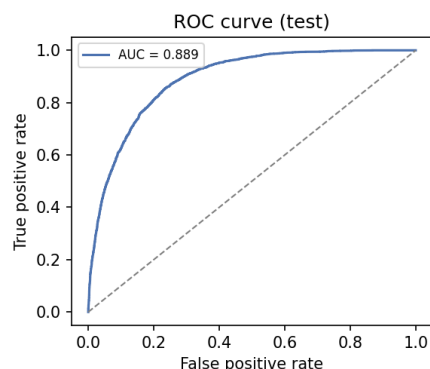


Figure 3. ROC curve for the selected model.

DISCUSSION

RandomForest is selected by mean CV F1=0.6641, but its advantage over LogisticRegression (CV F1=0.6602) is small (delta=0.0039). The two models therefore have similar overall discriminative ability, while their operating points differ. LogisticRegression has higher minority recall (0.849 versus 0.715) but lower precision (0.540 versus 0.620), so it would be preferable when missed positive cases are especially costly. RandomForest offers a more balanced precision-recall result and is the selected default under the F1 criterion. GradientBoosting obtains the highest accuracy and ROC-AUC, yet its lower recall reduces F1; this illustrates why accuracy alone would be a poor selection rule under imbalance. The test F1 of 0.6631 is close to the CV F1 of 0.6641, providing no obvious sign of overfitting. A limitation is that the hyperparameters are sensible defaults rather than tuned values; future work could evaluate threshold selection or nested tuning inside cross-validation.

CONCLUSION

The SKILL produces a reproducible classification workflow that handles missing values and mixed numeric and categorical predictors without leakage. RandomForest is the best model under the stated mean-CV-F1 criterion, although LogisticRegression is a credible alternative when recall is prioritised. The held-out results remain close to cross-validation estimates, and the confusion matrix confirms useful detection of both classes. All reported values are generated from the structured profiling and evaluation outputs.

REFERENCES

- [1] Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.
 - [2] Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5-32.
 - [3] McKinney, W. (2010). Data Structures for Statistical Computing in Python. *Proceedings of the 9th Python in Science Conference*, 56-61.
- Name: LIN TONG
 Matric number: G2608825L
 Assignment: IN6227-Assignment-1, Variant-2
 Model (LLM): GPT-6
 LLM interface: Codex desktop
 GitHub: <https://github.com/tonglynn/in6227-tabular-classification-skill>

Reflection - IN6227 Assignment 1, Variant 2

Human oversight

I treated the generated files as drafts and checked the intermediate outputs before accepting the final report. I first reviewed `profile.json` and `preprocess_config.json` to make sure the target was `label`, that the missing values had been identified, and that the numeric and categorical columns were assigned sensible preprocessing. I then checked `results.json` and `run_manifest.json` after training. In particular, I checked that preprocessing was inside the sklearn pipeline, that cross-validation was stratified, and that the test set was used only after model selection.

I did not simply accept the first generated version. I compared the file structure with the assignment, noticed that the report needed to be generated from the official template, and checked that the report and the supporting JSON files described the same run. I also ran the validation script and the two generalization checks on the Breast Cancer and Iris datasets. Those tests mattered because the skill was supposed to work beyond the supplied dataset; a report that worked only for a hard-coded `label` column would not satisfy Variant 2.

I also reviewed the report layout against the supplied Word template. The report contains the required sections, the first two pages are the generated report, and the Reflection follows after those pages. Before accepting the result, I opened the final PDF and manually checked the VITA block. The GitHub URL shown there was the repository I intended to submit: <https://github.com/tonglynn/in6227-tabular-classification-skill>.

I accepted the result because the report numbers matched the structured outputs, the report stayed within the two-page limit, and the final PDF contained my name, matric number, model metadata, LLM interface, and repository link. If I ran the workflow again, I would spend more time tuning the models inside cross-validation and would include a clearer comparison of different classification thresholds.

One practical change I made during review was to keep the generated report and the Reflection together in the final submission PDF. The two-page restriction applies to the generated report; the Reflection is attached after those pages. I also checked the first page manually for the exact assignment name and variant, since a technically correct report would still be incomplete if the required submission metadata were missing.

Critical evaluation

The decision I questioned most was using mean cross-validation F1 as the main model-selection metric. It is a reasonable default because the classes are imbalanced and F1 considers both precision and recall, but it does not reflect every possible cost of an error.

For this dataset, RandomForest had the best CV F1 at 0.6641, while LogisticRegression was very close at 0.6602. LogisticRegression had much higher recall for the minority class (0.849 versus 0.715), although its precision was lower. If missing a positive case were more serious than raising a false alarm, I would choose LogisticRegression or tune the decision threshold instead of automatically accepting RandomForest. I therefore agree with F1 as a transparent default for this assignment, but I would not treat it as the only acceptable decision rule in a real application.

I also chose not to add SMOTE or extensive feature engineering. That kept the workflow easier to explain and avoided introducing another possible source of leakage. The trade-off is that the models use mostly sensible defaults rather than a carefully tuned search, so the result should be read as a reproducible baseline rather than a production solution.

Another decision I reviewed was the use of balanced class weights for LogisticRegression and RandomForest but not for GradientBoosting. This is defensible because GradientBoosting does not expose the same `class_weight` option, but it also means the comparison is not perfectly symmetrical. GradientBoosting achieved the highest accuracy and ROC-AUC while producing a weaker F1, which reinforced my decision to report several metrics instead of presenting accuracy alone. If this were a real modelling task, I would either tune all models more systematically or explain the business cost of each error before choosing the final metric.

Trustworthiness

I checked the outputs in more than one place instead of relying only on the narrative report. The class counts in `profile.json` are 23,645 no and 7,464 yes, which sum to 31,109 after the three rows with missing targets are removed. The confusion matrix in `results.json` is `[[8762, 1403], [901, 2267]]`; its entries sum to 13,333, the reported test-set size.

I manually opened the final PDF and checked the GitHub URL in the VITA section. I also compared the displayed RandomForest F1 values with `results.json`: the report shows CV F1 0.6641 and test F1 0.6631, which match the underlying values when rounded to four decimal places. These checks gave me confidence that the report was generated from the actual run rather than being filled in manually.

The confusion matrix provided a second useful check. The four values, 8,762 true negatives, 1,403 false positives, 901 false negatives, and 2,267 true positives, add up to 13,333 test examples. That agrees with `n_test` in the results file. I also checked that the report's discussion did not claim a dramatic advantage for RandomForest: the F1 difference from LogisticRegression is only about 0.004. These small checks helped me distinguish a report that is internally consistent from one that merely sounds plausible.